KOENIG

step forward



Core Spring Boot Annotations

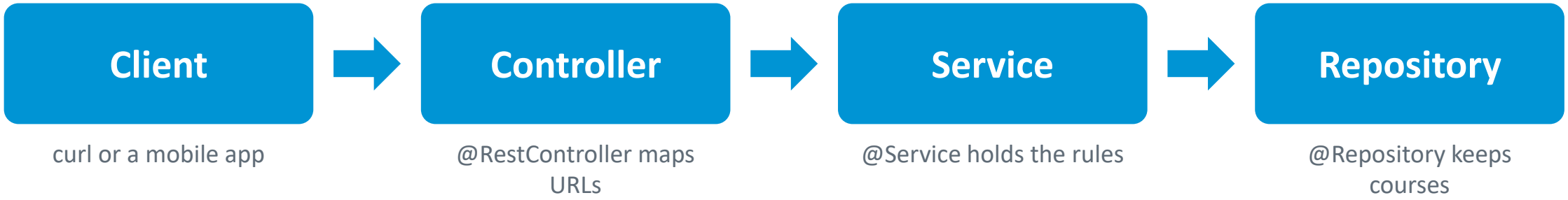
Spring Boot Fundamentals

What You Will Learn

1. Stereotypes recap, layers, @Controller vs @RestController
2. @SpringBootApplication and the component scan
3. @Import (recap: @Configuration and @Bean)
4. Request mapping annotations
5. @Async, @Scheduled and @Cacheable
6. Proxies: calls from outside only

Demo Scenario: Learnly Course API

Module 6 wired Learnly Academy's beans. Now the platform gets a REST API for its course catalogue, with background jobs and caching, all driven by annotations.



Annotations in action | **Routing** | **Background jobs** | **Caching**

Annotations: Labels on Code

- **Annotation:** metadata written with @ above a class, method or parameter.
- **Spring reads them** at startup and at runtime to decide what to do.
- **Declarative:** we say WHAT we want, the framework does the HOW.
- **Analogy:** luggage tags - the airport belt reads the tag, not the bag.

```java
@Service                          // class label
public class GreetingService {

    @Value("${app.greeting}")  // field label
    private String greeting;

    @Scheduled(fixedRate = 5000)  // method label
    public void announce() {
        System.out.println(greeting);
    }
}
```

Layers in Code

```java
@Repository
public class CourseRepository {
    public List<Course> findAll() { ... }
}
@Service
public class CourseService {
    private final CourseRepository repository;   // injected
    public CourseService(CourseRepository repository) {
        this.repository = repository;
    }
}
@RestController
public class CourseController {
    private final CourseService service;
    public CourseController(CourseService s) { ... }
}
```

- **Single constructor:** no @Autowired needed.
- **@Repository** also translates database exceptions to Spring's own.
- **Direction:** controller calls service, service calls repository.

Never skip a layer: a controller should not touch the repository directly

@Controller vs @RestController

@Controller

For classic web pages: methods return a view name such as "home" and a template renders HTML. Add @ResponseBody on a method to return data instead.

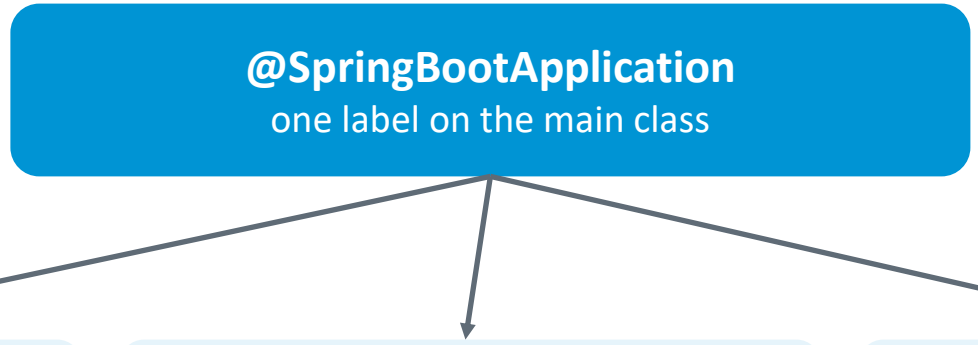
@RestController

= @Controller + @ResponseBody. Every method returns data (JSON) written straight into the response. Used for REST APIs.

```
@RestController                    // our choice in this course
public class HelloController {
    @GetMapping("/hello")
    public String hello() { return "Hello"; }    // body text
}
```

REST API = @RestController. HTML pages with templates = @Controller

@SpringBootApplication



@SpringBootConfiguration

This class is a configuration class; it can hold @Bean methods.

@EnableAutoConfiguration

Configure things automatically from the libraries on the classpath.

@ComponentScan

Scan this package and below for stereotype classes.

```
@SpringBootApplication
public class BootApp { ... }
```

What Each Part Does

- **@SpringBootConfiguration** a specialised @Configuration (found by tests too).
- **@EnableAutoConfiguration** reads the classpath: webmvc present means a web server and DispatcherServlet.
- **@ComponentScan** finds your @Component classes.

```
// Equivalent, written by hand
@SpringBootConfiguration
@EnableAutoConfiguration
@ComponentScan
public class BootApp {
    public static void main(String[] args) {
        SpringApplication.run(
            BootApp.class, args);
    }
}
```

@Import: Combine Configurations

- **@Import** pulls extra configuration classes into this one.
- **Why:** explicit control, or classes outside the scanned packages.
- **Also accepts** plain @Component classes.
- **Used by libraries** to switch features on, like @EnableAsync.

```java
@Configuration
public class SecurityConfig { ... }

@Configuration
public class MailConfig { ... }

@Configuration
@Import({SecurityConfig.class, MailConfig.class})
public class AppConfig { }
```

@RequestMapping and Shortcuts

- **@RequestMapping:** maps a URL (and method) to code.
- **On the class:** shared prefix for all methods.
- **Shortcuts:** @GetMapping, @PostMapping, @PutMapping, @DeleteMapping, @PatchMapping.
- **Each shortcut** = @RequestMapping with a fixed HTTP method.

```
@RestController
@RequestMapping("/courses")   // prefix
public class CourseController {
    @GetMapping                // GET
    public List<Course> list() { ... }
    @PostMapping               // POST
    public Course create(...) { ... }
}
```

HTTP Verbs and Annotations

GET - @GetMapping

Read data. Safe, no changes.

POST - @PostMapping

Create something new.

PUT / PATCH - @PutMapping / @PatchMapping

Replace or partly update.

DELETE - @DeleteMapping

Remove it.

Same URL, different verb, different action

@PathVariable and @RequestParam

- **@PathVariable:** a piece of the path - identifies WHICH resource.
- **@RequestParam:** ?key=value - filters, paging, options.
- **Optional:** required = false, or defaultValue.
- **Conversion:** text becomes long, enum, LocalDate automatically.

```
// GET /courses/7
@GetMapping("/{id}")
public Course one(@PathVariable long id) { ... }
// GET /courses?level=BEGINNER&limit=5
@GetMapping
public List<Course> list(
        @RequestParam(required = false) Level level,
        @RequestParam(defaultValue = "10") int limit) {
    ...
}
```

@RequestBody: JSON to Object

- **@RequestBody:** the HTTP body is converted into your object.
- **Records** make perfect request types.
- **Content-Type:** the client sends application/json.
- **Response:** the returned object becomes JSON too.

```
public record CreateCourseRequest(
        String title, Level level, BigDecimal price) { }
@PostMapping
@ResponseStatus(HttpStatus.CREATED)   // 201
public Course create(
        @RequestBody CreateCourseRequest request) {
    return service.add(request);
}
// body:  {"title":"Docker Basics",
//         "level":"BEGINNER","price":3499.00}
```

From URL to Method



- **Argument annotations** (@PathVariable, @RequestParam, @RequestBody) tell Spring where each value comes from.
- **Return value** is converted by message converters (JSON).

Demo: Routing with Mapping Annotations

What we build

- **Controller:** list with filters, one course, create and delete.
- **Requests:** sent with curl; the status code is part of the answer.
- **Mistakes:** an unknown id gives 404, bad input gives 400.

How the concept helps

- **@RequestMapping:** the shared prefix is written once.
- **Argument annotations:** path says which, query says how, body says what.
- **@ResponseStatus:** 201 for created, 204 for deleted.

Behavioural Annotations

@Transactional

all-or-nothing database work

@Async

run on another thread

@Scheduled

run on a timer

@Cacheable

remember a result

They add behaviour around a method without changing its code

@Async: Do It in the Background

- **Needs:** @EnableAsync on a configuration class.
- **Caller returns** at once; the method runs on a pool thread.
- **Return type:** void or CompletableFuture.
- **Name the executor:** @Async("notificationExecutor").

```
@Configuration
@EnableAsync
public class AsyncConfig { ... }

@Service
public class NotificationService {

    @Async("notificationExecutor")
    public void send(String to, String message) {
        // runs on thread "notify-1"
    }
}
```

@Scheduled: A Timer

- **Needs:** @EnableScheduling.
- **fixedRate:** every N ms from the previous start.
- **fixedDelay:** N ms after the previous run ends.
- **cron:** calendar times, e.g. 0 0 2 * * * for 2 am daily.
- **No parameters,** returns void.

```
@Component
public class HousekeepingJob {

    @Scheduled(initialDelay = 3000,
               fixedRateString = "${app.rate-ms:10000}")
    public void cleanUp() {
        // runs on thread "housekeeping-1"
    }

    @Scheduled(cron = "0 0 2 * * *")   // daily at 02:00
    public void nightly() { }
}
```

@Cacheable: Remember Results

- **Idea:** same input, same answer, skip the slow work.
- **Needs:** spring-boot-starter-cache and @EnableCaching.
- **Key:** method arguments by default.
- **@CacheEvict** removes entries when data changes.

```java
@Configuration
@EnableCaching
public class CacheConfig { }

@Service
public class PriceService {

    @Cacheable("prices")
    public double priceOf(String sku) {
        // slow call, executed once per sku
        return lookupInDatabase(sku);
    }
}
```

Demo: @Async, @Scheduled and @Cacheable

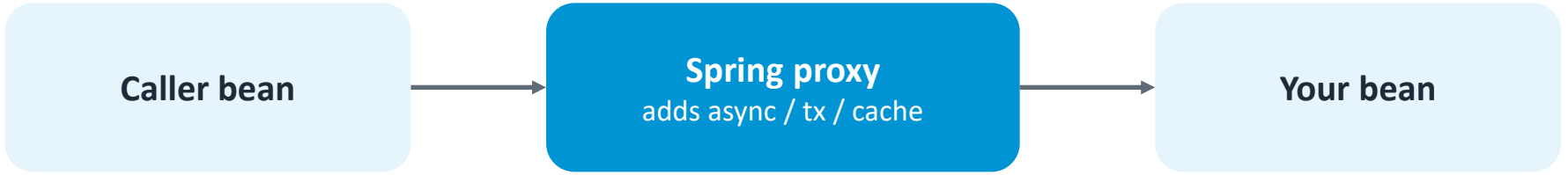
What we build

- **@Async:** the welcome mail runs on thread notify-1.
- **@Scheduled:** a job ticks every 200 ms by itself.
- **@Cacheable:** the slow price lookup runs once.

How the concept helps

- **Enable annotations:** without them nothing happens.
- **Proxy:** adds the behaviour around the method.
- **Evidence:** thread names and counters show what ran.

Proxies: Calls From Outside Only




```java
@Service
public class ReportService {
    public void run() {
        send();       // BROKEN: no proxy
    }
    @Async
    public void send() { }
}
```

- **Rule:** @Async, @Transactional and @Cacheable only work when another bean calls the method.
- **Fix:** move the annotated method into its own bean.
- **Also:** the method must be public and the class not final.

Calling an annotated method from the same class skips the behaviour

Demo: The Proxy Trap

What we build

- **Broken:** an @Async method called from its own class runs on main.
- **Fixed:** the method moves into its own bean.
- **Proof:** the thread name changes from main to notify-1.

How the concept helps

- **Proxy:** only calls that come from outside pass through it.
- **Same rule:** @Transactional and @Cacheable behave the same way.
- **Fix:** split the beans.

Annotation Cheat Sheet

Layer	@Controller @RestController @Service @Repository @Component
Setup	@SpringBootApplication @Configuration @Bean @Import
Web	@RequestMapping @GetMapping @PostMapping @PathVariable @RequestParam @RequestBody
Behaviour	@Async @Scheduled @Cacheable

Key Takeaways

1 **Stereotypes**

@Component is the base; @Service, @Repository, @Controller name the layer.

2 **Boot setup**

@SpringBootApplication = configuration + auto-configuration + scan.

3 **Routing**

Map URLs with Get/Post and argument annotations; Spring converts the input.

4 **Behaviour and proxies**

Enable @Async/@Scheduled/@EnableCaching and call the method from another bean.



Happy Learning :)